Hall Ticket: 2303A51929

Batch: 30

Assessment: 16.1

Task 1: Student Information System Schema

Prompt:

--Create a database schema named "Student Information System" with the following tables: "Students", "Courses", and "Enrollments". and define primary keys, foreign keys, and relationships

--Generate SQL queries for:

--1. Inserting a new student record

--2. Fetching all courses enrolled by a specific student

--3. Counting the number of students in each course

Code:

```
--Create a database schema named "Student Information System" with the following tables: "Students", "Courses", and "Enrollments". and define primary keys, foreign keys, and relationships
--Generate SQL queries for:
--1. Inserting a new student record
--2. Fetching all courses enrolled by a specific student
--3. Counting the number of students in each course

-- Create Students table
CREATE TABLE Students (
    StudentID INTEGER PRIMARY KEY AUTOINCREMENT,
    FirstName TEXT NOT NULL,
    LastName TEXT NOT NULL,
    DateOfBirth TEXT NOT NULL
);

-- Create Courses table
CREATE TABLE Courses (
    CourseID INTEGER PRIMARY KEY AUTOINCREMENT,
    CourseName TEXT NOT NULL,
    Credits INTEGER NOT NULL
);

-- Create Enrollments table
CREATE TABLE Enrollments (
    EnrollmentID INTEGER PRIMARY KEY AUTOINCREMENT,
    StudentID INTEGER,
    CourseID INTEGER,
    EnrollmentDate TEXT,
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);

-- Insert data
INSERT INTO Students (FirstName, LastName, DateOfBirth)
VALUES ('John', 'Doe', '2000-01-01');

INSERT INTO Courses (CourseName, Credits)
VALUES ('DBMS', 4), ('AI', 3);

INSERT INTO Enrollments (StudentID, CourseID, EnrollmentDate)
VALUES (1, 1, DATE('now')),
       (1, 2, DATE('now'));

-- Fetch courses of student
SELECT c.CourseName
FROM Enrollments e
JOIN Courses c ON e.CourseID = c.CourseID
WHERE e.StudentID = 1;

-- Count students in each course
SELECT c.CourseName, COUNT(e.StudentID) AS NumberOfStudents
FROM Courses c
LEFT JOIN Enrollments e ON c.CourseID = e.CourseID
GROUP BY c.CourseID;
```

Output:

Significance:

The Student Information System database helps in efficiently managing student and course details by organizing data into structured tables. It establishes relationships between students and courses using primary and foreign keys, which ensures data integrity and consistency. This system allows easy tracking of student enrollments and supports queries such as retrieving enrolled courses and counting students in each course.

Task 2: Hospital Management Database

Prompt:

--create table Tables: Doctors, Patients, Appointments.and define constraints (unique IDs, valid dates)

--generate SQL queries for:

--List all appointments for a specific doctor.

--Retrieve patient history by patient ID.

--Count total patients treated by each doctor.

-- Create Doctors table

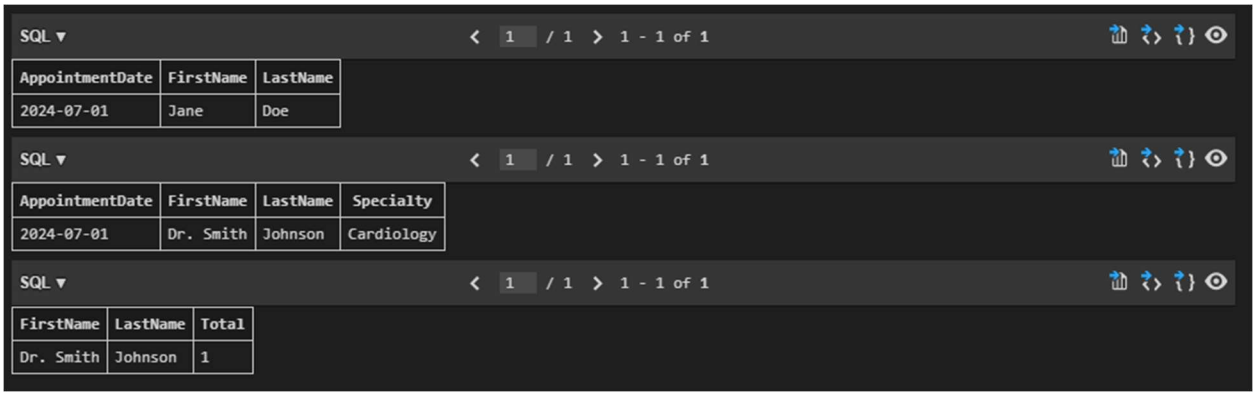
Code:

```sql
--create table Tables: Doctors, Patients, Appointments.and define constraints (unique IDs, valid dates)
--generate SQL queries for:
--List all appointments for a specific doctor.
--Retrieve patient history by patient ID.
--Count total patients treated by each doctor.
-- Create Doctors table
CREATE TABLE Doctors (
    DoctorID INTEGER PRIMARY KEY AUTOINCREMENT,
    FirstName TEXT NOT NULL,
    LastName TEXT NOT NULL,
    Specialty TEXT NOT NULL
);

-- Create Patients table
CREATE TABLE Patients (
    PatientID INTEGER PRIMARY KEY AUTOINCREMENT,
    FirstName TEXT NOT NULL,
    LastName TEXT NOT NULL,
    DateOfBirth TEXT NOT NULL
);

-- Create Appointments table
CREATE TABLE Appointments (
    AppointmentID INTEGER PRIMARY KEY AUTOINCREMENT,
    DoctorID INTEGER,
    PatientID INTEGER,
    AppointmentDate TEXT,
    FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID),
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID)
);
-- Insert data
INSERT INTO Doctors (FirstName, LastName, Specialty)
VALUES ('Dr. Smith', 'Johnson', 'Cardiology');
INSERT INTO Patients (FirstName, LastName, DateOfBirth)
VALUES ('Jane', 'Doe', '1990-05-15');
INSERT INTO Appointments (DoctorID, PatientID, AppointmentDate)
VALUES (1, 1, '2024-07-01');
-- List all appointments for a specific doctor
SELECT a.AppointmentDate, p.FirstName, p.LastName
FROM Appointments a
JOIN Patients p ON a.PatientID = p.PatientID
WHERE a.DoctorID = 1;
-- Retrieve patient history by patient ID
SELECT a.AppointmentDate, d.FirstName, d.LastName, d.Specialty
FROM Appointments a
JOIN Doctors d ON a.DoctorID = d.DoctorID
WHERE a.PatientID = 1;
-- Count total patients treated by each doctor
SELECT d.FirstName, d.LastName, COUNT(DISTINCT a.PatientID) AS Total
FROM Doctors d
LEFT JOIN Appointments a ON d.DoctorID = a.DoctorID
GROUP BY d.DoctorID;
```

Output:



Significance:

The Hospital Management System database is useful for maintaining records of doctors, patients, and appointments in an organized manner. It helps in tracking patient history and managing appointment schedules effectively. By establishing relationships between tables, the system ensures accurate data handling and enables analysis such as counting the number of patients treated by each doctor.

Task 3: Library Database

Prompt:

--create a table Books, Members, Loans.and suggest indexing strategy for faster queries.

--generate SQL queries for:

--Retrieve all books currently issued.

--Find overdue books (loan date > 30 days).

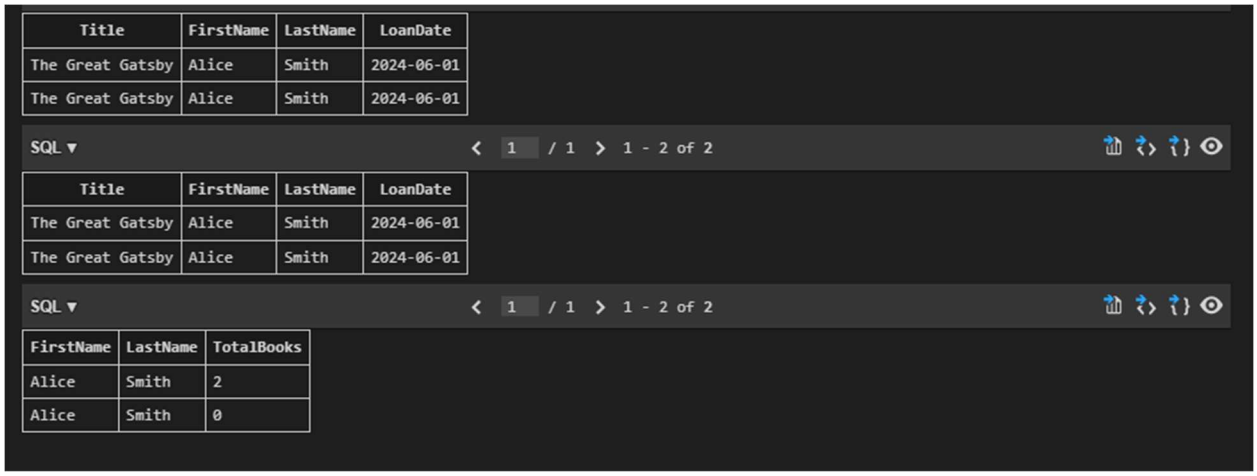
--Count number of books loaned by each member.

Code:

```sql
--create a table Books, Members, Loans.and suggest indexing strategy for faster queries.
--generate SQL queries for:
--Retrieve all books currently issued.
--Find overdue books (loan date > 30 days).
--Count number of books loaned by each member.

-- Create Books table
CREATE TABLE Books (
    BookID INTEGER PRIMARY KEY AUTOINCREMENT,
    Title TEXT NOT NULL,
    Author TEXT NOT NULL,
    PublishedYear INTEGER NOT NULL
);
-- Create Members table
CREATE TABLE Members (
    MemberID INTEGER PRIMARY KEY AUTOINCREMENT,
    FirstName TEXT NOT NULL,
    LastName TEXT NOT NULL,
    MembershipDate TEXT NOT NULL
);
-- Create Loans table
CREATE TABLE Loans (
    LoanID INTEGER PRIMARY KEY AUTOINCREMENT,
    BookID INTEGER,
    MemberID INTEGER,
    LoanDate TEXT,
    FOREIGN KEY (BookID) REFERENCES Books(BookID),
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
);
-- Insert data
INSERT INTO Books (Title, Author, PublishedYear)
VALUES ('The Great Gatsby', 'F. Scott Fitzgerald', 1925);
INSERT INTO Members (FirstName, LastName, MembershipDate)
VALUES ('Alice', 'Smith', '2020-01-01');
INSERT INTO Loans (BookID, MemberID, LoanDate)
VALUES (1, 1, '2024-06-01');
-- Retrieve all books currently issued
SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID;
-- Find overdue books (loan date > 30 days)
SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE DATE(l.LoanDate) < DATE('now', '-30 days');
-- Count number of books loaned by each member
SELECT m.FirstName, m.LastName, COUNT(l.BookID) AS TotalBooks
FROM Members m
LEFT JOIN Loans l ON m.MemberID = l.MemberID
GROUP BY m.MemberID;
```

Output:



Title	FirstName	LastName	LoanDate
The Great Gatsby	Alice	Smith	2024-06-01
The Great Gatsby	Alice	Smith	2024-06-01

SQL ▾ 1 / 1 1 - 2 of 2

Title	FirstName	LastName	LoanDate
The Great Gatsby	Alice	Smith	2024-06-01
The Great Gatsby	Alice	Smith	2024-06-01

SQL ▾ 1 / 1 1 - 2 of 2

FirstName	LastName	TotalBooks
Alice	Smith	2
Alice	Smith	0

Significance:

The Library Management System database helps in managing books, members, and loan records efficiently. It enables tracking of issued and overdue books, ensuring proper monitoring of library resources. The use of indexing improves query performance, making data retrieval faster. This system also supports analysis of borrowing patterns by counting the number of books loaned by each member.

Task 4: Real-Time Application: Online Shopping Database

Prompt:

-- create a Tables: Users, Products, Orders, OrderDetails.

--geneate SQL queries for:

--Retrieve all orders by a user.

--Find the most purchased product.

--Calculate total revenue in a given month.

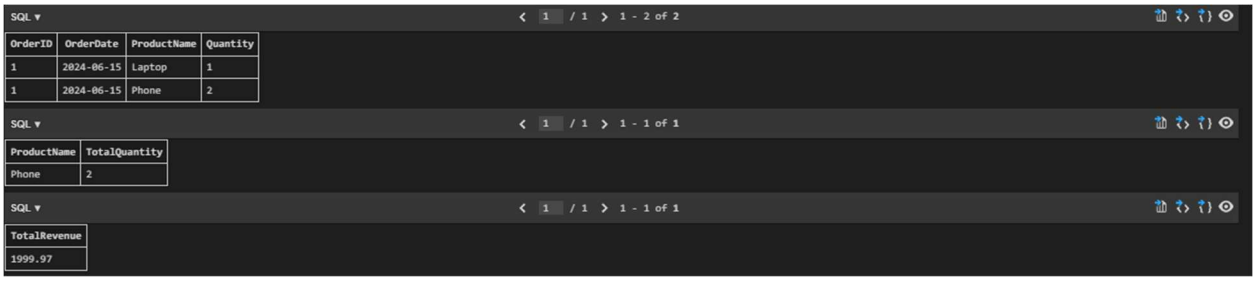
--suggest normalization improvements and query optimization.

Code:

```sql
-- create a Tables: Users, Products, Orders, OrderDetails.
--geneate SQL queries for:
--Retrieve all orders by a user.
--Find the most purchased product.
--Calculate total revenue in a given month.
--suggest normalization improvements and query optimization.
-- Create Users table
CREATE TABLE Users (

    UserID INTEGER PRIMARY KEY AUTOINCREMENT,
    FirstName TEXT NOT NULL,
    LastName TEXT NOT NULL,
    Email TEXT NOT NULL UNIQUE
);
-- Create Products table
CREATE TABLE Products (
    ProductID INTEGER PRIMARY KEY AUTOINCREMENT,
    ProductName TEXT NOT NULL,
    Price REAL NOT NULL
);
-- Create Orders table
CREATE TABLE Orders (
    OrderID INTEGER PRIMARY KEY AUTOINCREMENT,
    UserID INTEGER,
    OrderDate TEXT,
    FOREIGN KEY (UserID) REFERENCES Users(UserID)
);
-- Create OrderDetails table
CREATE TABLE OrderDetails (
    OrderDetailID INTEGER PRIMARY KEY AUTOINCREMENT,
    OrderID INTEGER,
    ProductID INTEGER,
    Quantity INTEGER,
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);
-- Insert data
INSERT INTO Users (FirstName, LastName, Email)
VALUES ('Bob', 'Brown', 'bob.brown@example.com');
INSERT INTO Products (ProductName, Price)
VALUES ('Laptop', 999.99), ('Phone', 499.99);
INSERT INTO Orders (UserID, OrderDate)
VALUES (1, '2024-06-15');
INSERT INTO OrderDetails (OrderID, ProductID, Quantity)
VALUES (1, 1, 1), (1, 2, 2);
-- Retrieve all orders by a user
SELECT o.OrderID, o.OrderDate, p.ProductName, od.Quantity
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
WHERE o.UserID = 1;
-- Find the most purchased product
SELECT p.ProductName, SUM(od.Quantity) AS TotalQuantity
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY od.ProductID
ORDER BY TotalQuantity DESC
LIMIT 1;
-- Calculate total revenue in a given month
SELECT SUM(p.Price * od.Quantity) AS TotalRevenue
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
WHERE strftime('%Y-%m', o.OrderDate) = '2024-06';
-- Normalization improvements:
-- 1. Ensure that the Products table is normalized to avoid redundancy (e.g., if there are multiple entries for the same product).
-- 2. Consider adding a separate table for Categories if products can belong to multiple categories.
-- Query optimization:
-- 1. Create indexes on foreign key columns (UserID in Orders, ProductID in OrderDetails) to speed up JOIN operations.
-- 2. Use EXPLAIN to analyze query performance and adjust indexes as needed.
```

Output:



Significance:

The Online Shopping database is important for handling user information, product details, and order transactions in an organized way. It supports tracking of customer purchases and helps in analyzing business data such as the most purchased product and total revenue. Proper normalization reduces data redundancy, and query optimization improves system performance, making it suitable for real-time applications.

Task 5: University Examination Database

Prompt:

--create table Students, Subjects, Exams, Results.and Define primary keys, foreign keys, and referential integrity constraints.

--generate SQL queries for:

--Insert examination results for a student.

--Retrieve subject-wise marks for a student.

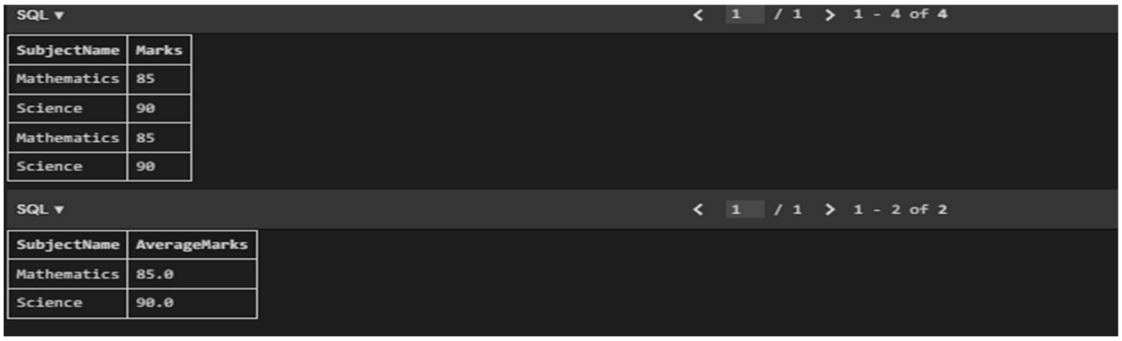
--Calculate average marks for each subject.

Code:

```sql
--create table Students, Subjects, Exams, Results.and Define primary keys, foreign keys, and referential integrity constraints.
--generate SQL queries for:
--Insert examination results for a student.
--Retrieve subject-wise marks for a student.
--Calculate average marks for each subject.

-- Create Students table
CREATE TABLE Students (
    StudentID INTEGER PRIMARY KEY AUTOINCREMENT,
    FirstName TEXT NOT NULL,
    LastName TEXT NOT NULL,
    DateOfBirth TEXT NOT NULL
);
-- Create Subjects table
CREATE TABLE Subjects (
    SubjectID INTEGER PRIMARY KEY AUTOINCREMENT,
    SubjectName TEXT NOT NULL
);
-- Create Exams table
CREATE TABLE Exams (
    ExamID INTEGER PRIMARY KEY AUTOINCREMENT,
    SubjectID INTEGER,
    ExamDate TEXT,
    FOREIGN KEY (SubjectID) REFERENCES Subjects(SubjectID)
);
-- Create Results table
CREATE TABLE Results (
    ResultID INTEGER PRIMARY KEY AUTOINCREMENT,
    StudentID INTEGER,
    ExamID INTEGER,
    Marks INTEGER,
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
    FOREIGN KEY (ExamID) REFERENCES Exams(ExamID)
);
-- Insert data
INSERT INTO Students (FirstName, LastName, DateOfBirth)
VALUES ('Emily', 'Clark', '2001-02-20');
INSERT INTO Subjects (SubjectName)
VALUES ('Mathematics'), ('Science');
INSERT INTO Exams (SubjectID, ExamDate)
VALUES (1, '2024-06-01'), (2, '2024-06-02');
INSERT INTO Results (StudentID, ExamID, Marks)
VALUES (1, 1, 85), (1, 2, 90);
-- Insert examination results for a student
INSERT INTO Results (StudentID, ExamID, Marks)
VALUES (1, 1, 85), (1, 2, 90);
-- Retrieve subject-wise marks for a student
SELECT s.SubjectName, r.Marks
FROM Results r
JOIN Exams e ON r.ExamID = e.ExamID
JOIN Subjects s ON e.SubjectID = s.SubjectID
WHERE r.StudentID = 1;
-- Calculate average marks for each subject
SELECT s.SubjectName, AVG(r.Marks) AS AverageMarks
FROM Results r
JOIN Exams e ON r.ExamID = e.ExamID
JOIN Subjects s ON e.SubjectID = s.SubjectID
GROUP BY s.SubjectID;
```

Output:



Significance:

The University Examination System database is designed to store and manage student, subject, exam, and result information efficiently. It ensures data accuracy through referential integrity and structured relationships. This system helps in evaluating student performance by retrieving subject-wise marks and calculating average scores, which is useful for academic analysis.